

بخش الف)

1. نوشتن کد VHDL

یک فایل منبع VHDL شامل entity و architecture ایجاد شد. در بخش architecture، انتخاب ورودی‌ها بر اساس وضعیت سه بیت S پیاده‌سازی شد (با استفاده از case یا with/select).

2. تعریف پایه‌ها با فایل UCF

پایه‌های فیزیکی FPGA برای ورودی‌ها، انتخاب‌ها و خروجی LED مطابق توضیحات تمرین تنظیم شدند.

3. پیاده‌سازی (Implement Design) و سنتز (Synthesize)

پروژه در ISE سنتز شد و Route & Map/Place بدون خطا انجام شد.

4. تست بنچ و شبیه‌سازی در ISim

یک فایل TestBench نوشته شد که در آن تمام حالت‌های ترکیبی S0..S2 اعمال و خروجی بررسی شد.

نتایج شبیه‌سازی نشان داد خروجی Q0 مطابق ورودی انتخاب‌شده تغییر می‌کند.

بخش ب)

هدف این بود که با استفاده از مالتی‌پلکسر 1×8 نوشته‌شده در بخش الف، یک مالتی‌پلکسر 1×16 طراحی کنیم. این کار دقیقاً نمونه‌ای از طراحی سلسله‌مراتبی (Hierarchical Design) در VHDL است و نشان می‌دهد چطور می‌توان یک بلوک کوچکتر را در مدارهای بزرگتر دوباره به‌کار گرفت.

۱. تعریف component (از 8×1 MUX گرفته شده از بخش الف)

ابتدا در ابتدای فایل VHDL بخش ب، ساختار مالتی‌پلکسر 1×8 که قبلاً نوشته بودیم، به‌عنوان یک Component معرفی شد. این component دقیقاً شامل همان ورودی‌ها (I0..I7)، پایه‌های انتخاب (S0..S2) و خروجی Q0 بود.

به این صورت VHDL می‌فهمد که یک بلوک از پیش‌تعریف‌شده داریم که می‌توانیم هر تعداد بار از آن استفاده کنیم.

۲. نمونه‌سازی (Instantiation) دو مالتی‌پلکسر 1×8

برای ساخت یک مالتکسپلر 1×16 نیاز داریم ورودی‌های ۱۶ تایی را به دو گروه تقسیم کنیم:

ورودی‌های I0 تا I7 → وصل به MUX اول

ورودی‌های I8 تا I15 → وصل به MUX دوم

بنابراین از component ایجادشده در مرحله قبل دو بار نمونه‌سازی شد.

هر نمونه دقیقاً همان MUX بخش الف است که این‌بار فقط ورودی‌های متفاوت دریافت می‌کند.

این یعنی کل منطق مالتی‌پلکسر 1×8 در بخش ب دوباره نوشته نشد، فقط استفاده شد.

۳. استفاده از بیت انتخاب چهارم برای انتخاب بین دو خروجی

حالا باید تعیین کنیم کدام‌یک از این دو خروجی، باید خروجی نهایی Q0 باشد.

اینجا از بیت انتخاب چهارم یعنی S3 استفاده کردیم:

اگر $S3 = 0$ → ورودی‌ها در بازه 0 تا 7 انتخاب شوند → خروجی از MUX اول بیاید

اگر $S3 = 1$ → ورودی‌ها در بازه 8 تا 15 انتخاب شوند → خروجی از MUX دوم بیاید

این بخش با یک 1×2 MUX ساده نوشته شد (که در VHDL با یک if یا select پیاده‌سازی شد).

۴. نتیجه: ساخت یک 16×1 MUX بدون دوبار منویسی منطقی

در واقع در بخش ب منطق داخلی یک مالتکسپلر 1×16 را از صفر پیاده‌سازی نکردیم؛

بلکه:

- کد بخش الف را به‌عنوان یک واحد قابل‌استفاده مجدد (Reusable Block) استفاده کردیم
- با قرار دادن دو نمونه از آن و اضافه‌کردن فقط یک انتخاب‌کننده ساده، مالتی‌پلکسر 1×16 ساخته شد